KolmoX: Structural Preconditioning Architecture for Lossless Heterogeneous Data Compression

Transcending the Limits of Black-Box Entropy Coders via Bit-Exact Domain Decomposition

Author: KolmoX Open-Source Research Project

Date: August 2026

Specification: KMX2 Engine Spec (v1.2.1)

Correction notice - 2026-09-03. *The benchmark table and abstract of this document previously reported figures that could not be reproduced by the shipped code. Several described transforms that were not reachable from the domain router at all, and the numbers had been written into the document by a text-substitution script rather than measured. All eleven rows have since been re-measured against the current pipeline with bit-exact roundtrip assertions. Ten of the eleven use synthetic datasets and are marked †; only the Astrophysics FITS row rests on a real-world production capture. See the repository README for the same table and the methodology notes.*

Abstract

Conventional general-purpose compression algorithms (e.g., LZMA, Deflate, Zstandard) treat arbitrary inputs as untyped, one-dimensional scalar byte streams. While remarkably effective for textual data exhibiting repeated substring sequences, they systematically underperform on structured numerical, spatial, and floating-point domains—such as continuous trajectories, telemetry, multidimensional scientific floats, raster frames, and audio waveforms—due to high local entropy in lower-order bit planes.

In this paper, we present **KolmoX**, a two-stage hierarchical lossless compression architecture based on structural preconditioning. In the primary stage, a domain-aware engine identifies payload schemas and executes deterministic, bit-exact transforms: columnar channel demuxing, IEEE-754 *byte-plane slicing*, stride autocorrelation, and 2D spatial delta modeling. In the secondary stage, the separated streams are encapsulated into the multi-stream **KMX2** container and encoded via Finite State Entropy (FSE) using Zstandard.

Empirical evaluations across 11 distinct domains demonstrate consistent gains over baseline state-of-the-art compressors wherever the payload actually carries the structure a transform exploits. Measured end-to-end through the public pipeline API, KolmoX achieves **+64.10%** net reduction over Zstandard on CNC toolpath G-Code (15.84x ratio), **+63.12%** on a tessellated parametric CAD mesh (16.27x ratio), **+60.48%** on stereo PCM audio (2.59x ratio), and **+54.67%** on fixed-stride binary register frames, with decompression throughput reaching **~1094 MB/s** on the binary register path. These figures were obtained on synthetically generated datasets, disclosed as such in the table below. On the one real-world production corpus evaluated - a 117 MB JWST MIRI observation - the measured gain is **+16.31%** (1.47x → 1.76x).

Crucially, the architecture is designed to never lose to its own baseline: every domain transform competes against a plain Zstandard encoding of the same input, and the smaller of the two is what gets stored. On a high-entropy payload where no structure exists to exploit, the transform is discarded and the cost collapses to the 24-byte KMX2 header (measured: -0.46% on an x86 executable).

Comprehensive Benchmark Results

Every row was measured on 2026-09-03 through the public `KolmoXPipeline.compress_bytes()` / `decompress_bytes()` API - the path an actual user exercises - with a bit-exact roundtrip assertion on each. Figures obtained by invoking engine classes directly, outside the pipeline, are deliberately excluded: they omit the 24-byte KMX2 container header and bypass the adaptive competitive fallback, and were the source of the discrepancies this revision corrects. Every † row is reproducible by third parties with `python benchmarks/benchmark_extended.py`, which regenerates each dataset from a fixed seed and re-asserts every roundtrip. Compression ratios are deterministic; the throughput columns are single-run samples and vary by roughly ±15% between runs.

Data Domain	Transformation Pipeline	Baseline (Zstd L3)	KolmoX (KMX2)	Gain vs Zstd	Throughput (Comp / Decomp)
CNC G-Code (.gcode) †	Columnar Axis Separation	5.69x	**15.84x**	**+64.10%**	~24 MB/s / ~35 MB/s
Parametric 3D CAD Mesh (.obj) †	Prefix-Grouped Vertex Plane Slicing	6.00x	**16.27x**	**+63.12%**	~40 MB/s / ~80 MB/s
Audio PCM 16-bit (.wav) †	Stereo Mid/Side Decorrelation	1.02x	**2.59x**	**+60.48%**	~172 MB/s / ~284 MB/s
LiDAR XYZ Point Cloud † [1]	Columnar Coordinate Slicing	27.11x	**65.79x**	**+58.79%**	~80 MB/s / ~106 MB/s
Binary Register Packets (.bin) †	Stride Autocorr + Demux	1.86x	**4.10x**	**+54.67%**	~158 MB/s / ~1094 MB/s
Industrial Telemetry (.csv) †	Shape-Grouped Columnar Demux	4.59x	**8.23x**	**+44.24%**	~47 MB/s / ~98 MB/s
Temporal Video Sequence (.kmxvraw) †	Frame XOR Delta (SIMD)	1.00x	**1.77x**	**+43.45%**	~120 MB/s / ~189 MB/s
2D Natural Sensor Raster (.bmp) †	2D Spatial Delta	1.07x	**1.80x**	**+40.38%**	~37 MB/s / ~2.4 MB/s
Dense Float32 Buffer (250k values) †	IEEE-754 Byte-Plane Slicing	1.29x	**1.89x**	**+31.86%**	~236 MB/s / ~642 MB/s
Astrophysics FITS (JWST) - real data	IEEE-754 Byte-Plane Slicing (auto-routed)	1.47x	**1.76x**	**+16.31%**	~213 MB/s / ~714 MB/s
x86 Binary Executable (.exe) † [2]	Adaptive Fallback (BCJ/Zstd)	7.59x	**7.56x**	**-0.46%**	~13 MB/s / ~879 MB/s

† Synthetic dataset, generated programmatically. Only the Astrophysics FITS row uses a real-world production capture: a 117 MB NASA/STScI JWST MIRI observation of the Carina Nebula, compressed whole-file through the automatic domain router. Compressing only the extracted pixel plane instead yields 1.26x → 1.61x (+21.41%).

*[1] The LiDAR dataset is a deterministic arithmetic ramp, which is why even the plain Zstd baseline reaches 27.11x. It demonstrates that the transform functions, but it is **not** representative of real scanner output and should be read as a mechanism check rather than a performance claim.*

[2] A 40 KB synthetic instruction stream; at that size the throughput timings are dominated by measurement noise.

Note: Domain gains depend on the input actually carrying the structure the transform exploits. On a high-entropy payload - a random-walk mesh with six decimals of noise per coordinate, or the x86 stream above - the transform yields nothing, the adaptive competitive fallback discards it, and the stored result is the plain Zstd baseline plus the 24-byte KMX2 header. The raster decompression figure (~2.4 MB/s) reflects a pure-Python pixel loop whose sequential dependency has not yet been ported to the C extension; it is the current honest number, not a target.

Domain Transform Specifications & Formal Theory

1. Astrophysics FITS & Multidimensional Tensor Modeling

Astronomical instruments (e.g., JWST NIRCam/MIRI, Hubble STIS) produce FITS data cubes formatted in Big-Endian standard byte order with 2880-byte header blocks followed by dense floating-point (`>f4`) or integer (`>i4`) matrices.

What the pipeline actually does today. A `.fits` input is classified by `DomainRouter.detect_domain()` as the generic `FLOAT32` domain and preconditioned by `ScientificFloatEngine.transform_f32_byte_plane()`. That transform is a single stage - a 4-plane byte transposition applied uniformly across the whole file, headers included:

$$\mathcal{M}_{i,j} \xrightarrow{\text{slicing}} \{P_0(i), P_1(i), P_2(i), P_3(i)\}$$

Floating-point matrices thereby isolate the sign bit and exponent (high byte P_3) from the least significant mantissa bits (P_0, P_1). Because physical sky backgrounds exhibit localized thermal equilibrium, P_3 and P_2 collapse into long run-length sequences, allowing entropy coders to process uniform statistical distributions. It is worth being precise about the limits of this path: it applies no spatial delta, performs no endianness handling, and has no awareness of FITS structure - it treats the file as an undifferentiated 4-byte-aligned buffer. The **+16.31%** reported in the table above is what this generic path achieves on a real JWST observation.

A dedicated engine exists but is not yet routed. The codebase also contains a `FitsEngine` class (`kolmox.engines.extended_domains`) implementing the richer two-stage scheme this section previously described as if it were the active path: a per-HDU horizontal modular delta,

$$\Delta_{x,y} = (S_{x,y} - S_{x-1,y}) \pmod{2^{32}}$$

followed by byte-plane slicing of the residuals, with `astropy`-based HDU parsing and its own serialization format. It is covered by unit tests, but `DomainRouter` never dispatches to it, and it does not emit a KMX2 container - so no figure in this paper is produced by it. Wiring it into the router, and measuring whether the delta stage actually improves on the generic transposition for astronomical data, is open work.

2. High-Throughput In-Memory Vector Slicing

In dense vector environments (e.g., neural embeddings, high-frequency telemetry caching), float representations incur severe penalties under LZ77-based match finders due to the chaotic nature of low-order mantissa bits. KolmoX transposes contiguous 32-bit buffers into 4 disjoint orthogonal memory blocks, isolating high-entropy IEEE-754 mantissa noise. Measured end-to-end through the pipeline, this path sustains **~642 MB/s** decompression on a 250k-value synthetic float buffer and **~714 MB/s** on the 117 MB JWST observation; the highest figure recorded across all domains is **~1094 MB/s**, on the fixed-stride binary register path.

KMX2 Container Specification

The **KMX2** container format implements a fixed 24-byte header supporting multi-stream encapsulation:

```
Header = < Magic(4B), Version(2B), DomainID(1B), Flags(1B), RawSize(8B), AuxLen(8B) >
```